

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the working principle of an Artificial Neuron and implement a Single Layer Perceptron from scratch for binary classification. The Banknote Authentication dataset is used to train and evaluate the perceptron model. The experiment also aims to study the effect of feature normalization, analyse the learning behaviour using different plots and evaluate the classifier using standard performance metrics such as Accuracy, Precision, Recall and F1-score.

2. Background Theory

Artificial Neural Networks are computational models inspired by the functioning of biological neurons. The fundamental processing unit of an Artificial Neural Network is the Artificial Neuron. Each neuron receives several input features, multiplies them with trainable weights, adds a bias value and passes the result through an activation function to generate an output.

The Single Layer Perceptron proposed by Frank Rosenblatt is one of the earliest supervised learning algorithms. It is capable of solving only linearly separable binary classification problems. During training, the perceptron iteratively updates its weights and bias using the perceptron learning rule until the classification error is minimized.

Mathematical Formulation

The weighted sum of the inputs is

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$ denotes the input feature vector.
- $\mathbf{w}$ denotes the weight vector.
- b denotes the bias.
- z denotes the weighted sum.

The output of the perceptron is

$$\hat{y} = f(z)$$

where $f(z)$ represents the activation function.

The perceptron learning rule updates the weights according to

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

The bias is updated using

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where

- η is the learning rate.
- y is the actual class.
- $\hat{y}$ is the predicted class.

3. Activation Function

An activation function determines the output of a neuron based on its weighted input. In this experiment, the Step Activation Function is employed because the problem involves binary classification. The function produces either 0 or 1 depending on the value of the weighted sum.

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function is simple and computationally efficient. However, it is not differentiable and therefore cannot be used with gradient-based optimization algorithms such as backpropagation.

Activation Function	Output Range	Typical Application
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers
ReLU	$[0, \infty)$	Deep Neural Networks
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification

Although several activation functions exist, only the Step Activation Function is used in this experiment because the Single Layer Perceptron performs binary classification on linearly separable data.

4. Dataset Description

The experiment uses the **Banknote Authentication Dataset** obtained from the UCI Machine Learning Repository. The dataset consists of statistical features extracted from digital images of genuine and forged banknotes using wavelet transforms.

Property	Value
Dataset Name	Banknote Authentication Dataset
Source	UCI Machine Learning Repository
Total Samples	1372
Input Features	4
Target Classes	2
Missing Values	None
Learning Type	Supervised Learning
Problem Type	Binary Classification

Feature Description

Feature	Description
Variance	Variance of the wavelet transformed image.
Skewness	Degree of asymmetry in the image distribution.
Curtosis	Sharpness or flatness of the probability distribution.
Entropy	Measure of randomness present in the image.

Target Classes

- Class 0 – Authentic Banknote
- Class 1 – Forged Banknote

The dataset contains no missing values and is well suited for demonstrating the learning capability of a Single Layer Perceptron after feature normalization.

5. Experimental Procedure

The experiment was carried out using Python in Google Colab. The implementation followed a systematic workflow beginning from dataset loading to model evaluation.

Task 1: Dataset Exploration

The dataset was first imported into the notebook using the Pandas library. The first five samples, dataset dimensions, missing values and descriptive statistics were obtained.

Output

- Total Samples : 1372
- Total Features : 4
- Target Classes : 2
- Missing Values : None

The descriptive statistics indicated that the dataset contains features with different numerical ranges. Therefore, feature normalization was performed before training the perceptron model.

Task 2: Exploratory Data Analysis

Exploratory Data Analysis (EDA) was carried out to understand the characteristics of the dataset before model training.

The following visualizations were generated.

- Feature Histograms
- Correlation Heatmap
- Scatter Plot
- Box Plot

These visualizations helped in identifying feature distributions, relationships among variables and the presence of outliers.

Task 3: Data Preprocessing

Before training, the dataset was preprocessed using the following steps.

1. Selected the four numerical features as input variables.
2. Selected the class column as the target variable.
3. Normalized all feature values using Min-Max Scaling.
4. Split the dataset into training and testing subsets.
5. Used an 80:20 train-test ratio.

The normalized dataset ensured that all features contributed equally during weight updates.

Task 4: Perceptron Implementation

A Single Layer Perceptron was implemented completely from scratch without using any machine learning libraries.

The implementation consisted of

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule
- Weight Update
- Bias Update

The weights were initialized with small random values and updated after every misclassified sample according to the perceptron learning rule.

Task 5: Model Training

The perceptron was trained for a maximum of 100 epochs.

During every epoch,

- Forward propagation was performed.
- Misclassified samples were identified.
- Weights were updated.
- Bias was updated.
- Training error was recorded.

Training continued until all epochs were completed while monitoring convergence through the number of misclassified samples.

Task 6: Model Evaluation

After training, the classifier was evaluated using

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

The evaluation metrics measured the effectiveness of the trained perceptron on unseen testing samples.

6. Source Code

The complete implementation was carried out in Python using Google Colab.

https://github.com/K-Mithran/Deep-Learning/blob/main/Lab1/DL_1.ipynb

The major stages of implementation included

1. Loading the dataset.
2. Performing exploratory data analysis.
3. Normalizing the dataset.
4. Splitting the dataset.
5. Implementing the perceptron from scratch.
6. Training the perceptron.
7. Evaluating the trained model.
8. Visualizing the learning behaviour.

The complete source code used in this experiment is available in the submitted Google Colab notebook and GitHub repository.

7. Experimental Results

7.1 Dataset Exploration

The Banknote Authentication dataset was successfully loaded into Google Colab. The dataset contains four numerical input features and one binary target class.

Property	Observed Value
Dataset Size	1372 Samples
Number of Features	4
Number of Classes	2
Missing Values	0
Training Samples	1097
Testing Samples	275

The dataset was found to be complete with no missing values. The descriptive statistics showed noticeable differences in feature ranges, making feature normalization necessary before training.

7.2 Data Preprocessing Results

After applying Min-Max Normalization, every feature value was transformed into the range $[0,1]$.

The dataset was divided into

- Training Set : 80%
- Testing Set : 20%

This preprocessing step improved convergence during perceptron training.

7.3 Model Training

The perceptron was trained for 100 epochs.

During training, the number of misclassified samples gradually reduced while the weights and bias converged to stable values.

The final learned parameters are shown below.

Parameter	Value
Weight 1	-0.2301
Weight 2	-0.2384
Weight 3	-0.2603
Weight 4	0.0007
Bias	0.31
Maximum Epochs	100

7.4 Performance Metrics

The trained perceptron was evaluated on the testing dataset using standard classification metrics.

Metric	Value
Accuracy	98.18%
Precision	100.00%
Recall	96.03%
F1-Score	97.98%

The obtained results indicate that the perceptron successfully classified the majority of unseen testing samples. The perfect precision value indicates that every banknote predicted as forged was indeed forged, while the high recall confirms that almost all forged banknotes were correctly identified.

7.5 Learning Rate Comparison

The perceptron model was trained using three different learning rates to study their influence on model performance.

Learning Rate	Epochs	Accuracy
0.001	100	99.64%
0.010	100	99.64%
0.100	100	98.91%

The comparison shows that learning rates of 0.001 and 0.01 achieved the highest classification accuracy. Increasing the learning rate to 0.1 resulted in a slight reduction in performance because of larger parameter updates during training.

8. Performance Summary

Dataset Size	1372
Training Samples	1097
Testing Samples	275
Features	4
Learning Rate	0.01
Epochs	100
Final Bias	0.31
Accuracy	98.18%
Precision	100.00%
Recall	96.03%
F1-Score	97.98%

9. Generated Plots with Interpretation

9.1 Feature Histograms

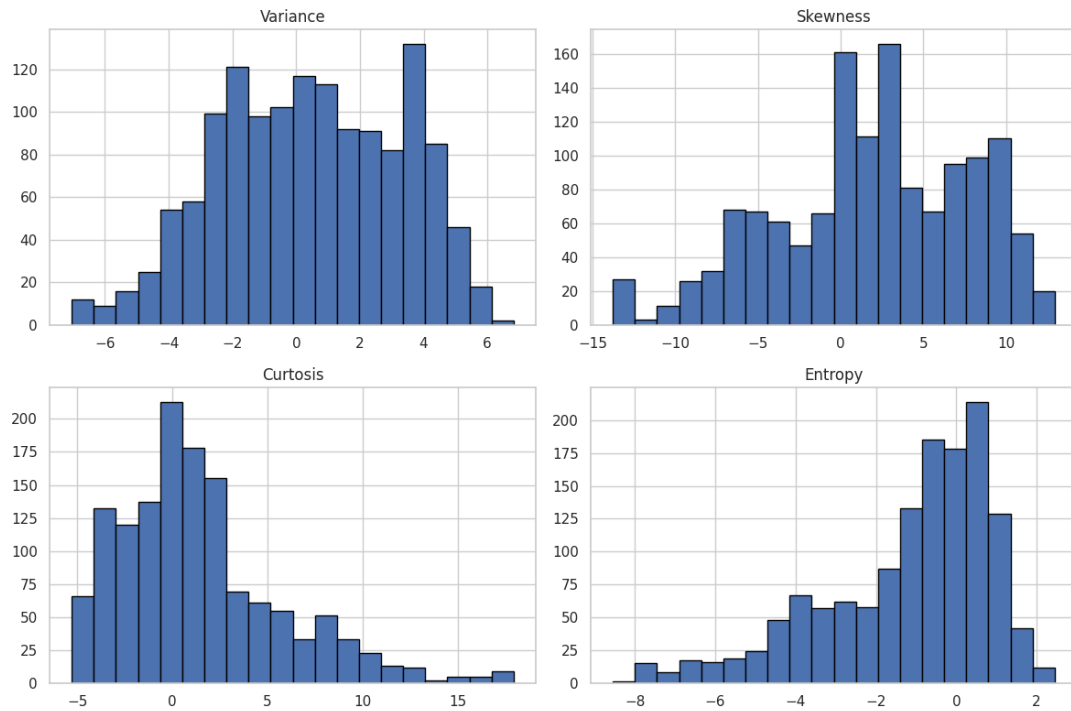


Figure 1: Feature Histograms of the Banknote Authentication Dataset

The histograms illustrate the distribution of all four numerical features present in the dataset. The features exhibit different ranges and varying degrees of skewness, indicating that normalization is necessary before training the perceptron.

9.2 Correlation Heatmap

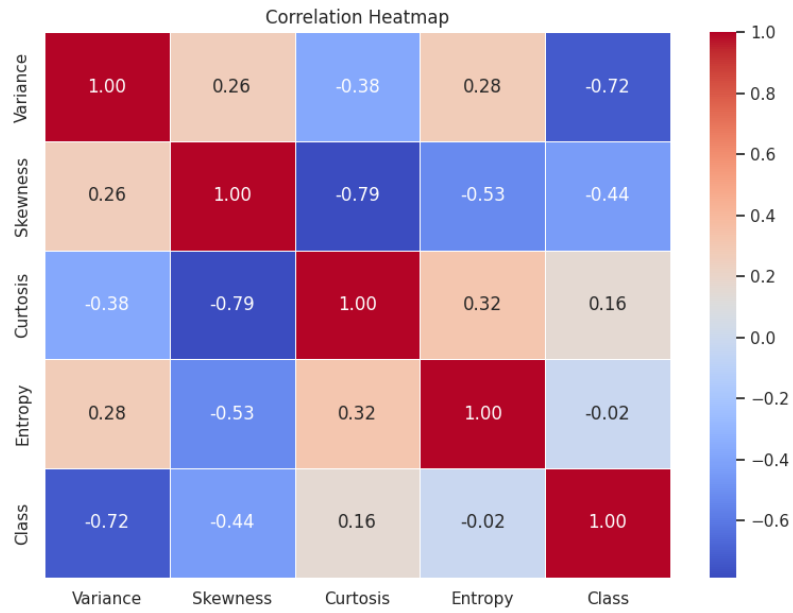


Figure 2: Correlation Heatmap

The heatmap illustrates the correlation between the input features. Highly correlated features provide useful information regarding the relationships among variables and help in understanding the overall structure of the dataset.

9.3 Scatter Plot

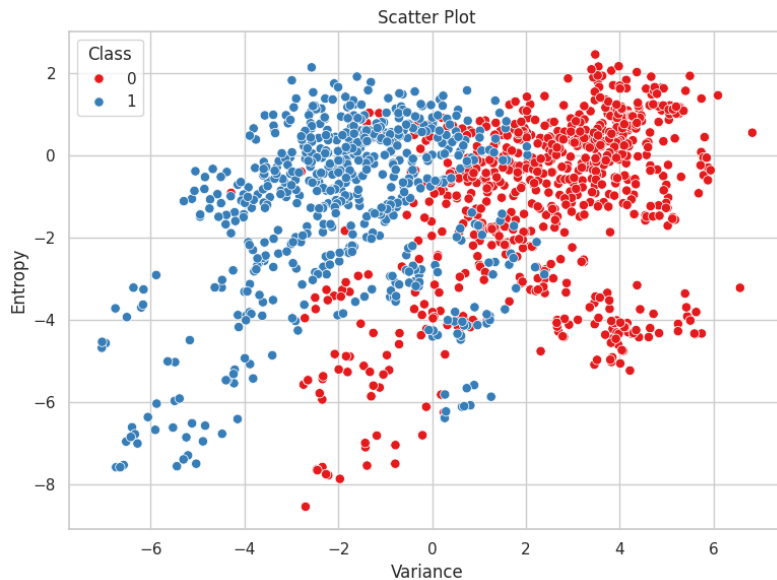


Figure 3: Scatter Plot

The scatter plot shows that the two classes are largely separable using the selected features. This justifies the use of a Single Layer Perceptron, which performs well on linearly separable datasets.

9.4 Box Plot

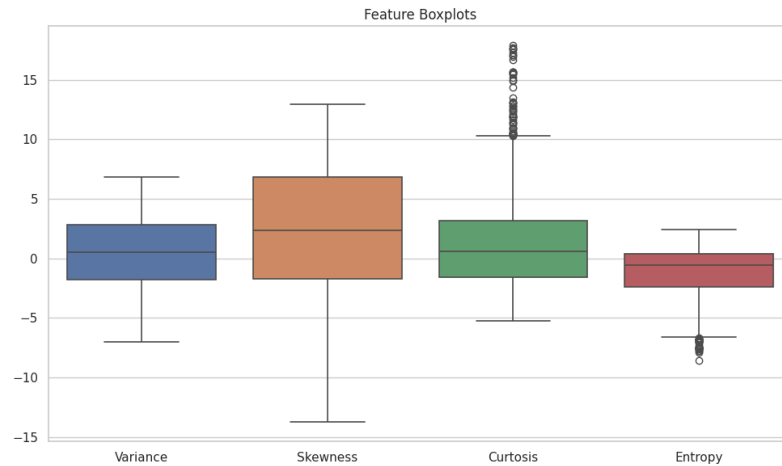


Figure 4: Box Plot

The box plot summarizes the spread of each feature and highlights the presence of a few outliers. Since Min-Max Normalization was applied before training, these outliers did not significantly affect the learning process.

9.5 Confusion Matrix

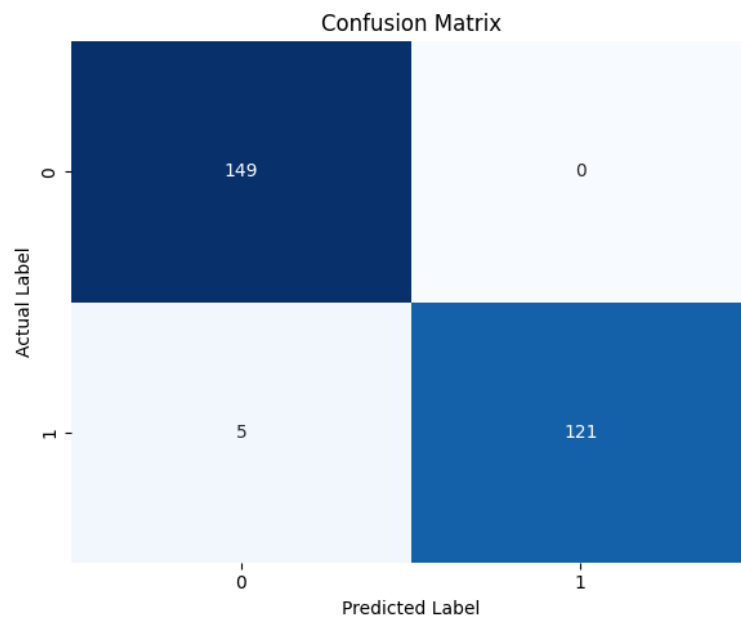


Figure 5: Confusion Matrix

The confusion matrix indicates that the perceptron correctly classified almost every testing sample. The small number of false negatives demonstrates the effectiveness of the trained model in distinguishing authentic and forged banknotes.

9.6 Training Error vs Epoch

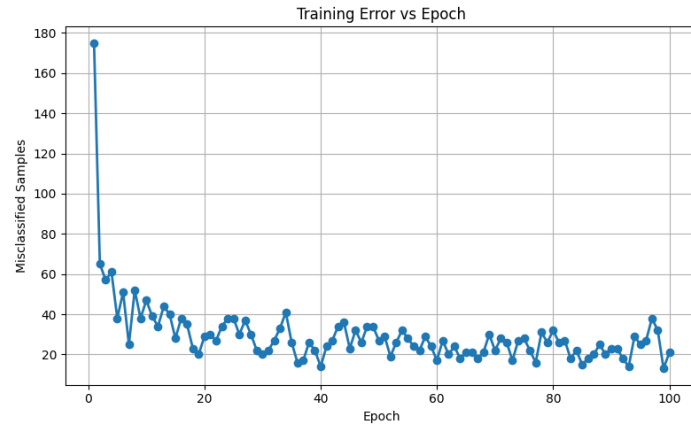


Figure 6: Training Error vs Epoch

The number of misclassified samples decreases considerably as training progresses. This behaviour indicates that the perceptron successfully learns a suitable decision boundary and gradually converges towards a stable solution.

9.7 Weight Evolution During Training

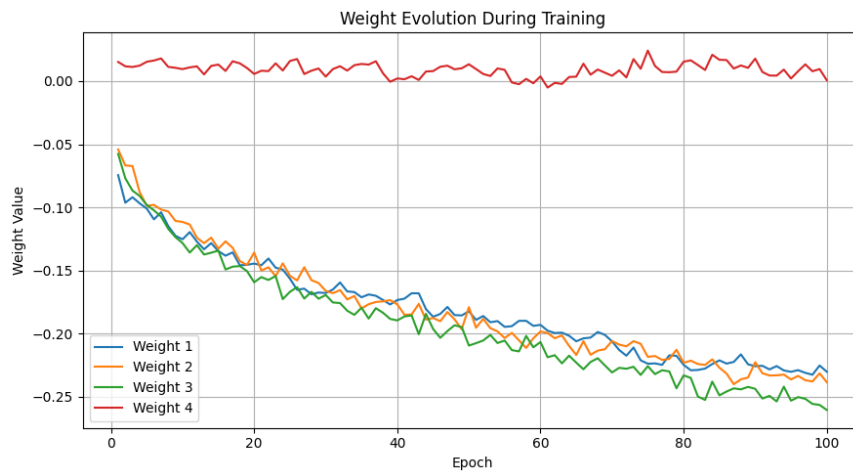


Figure 7: Weight Evolution During Training

The weight evaluation graph illustrates how the four weights are continuously adjusted during learning. Towards the final epochs the weights stabilize, indicating convergence of the learning algorithm.

9.8 Bias Evaluation

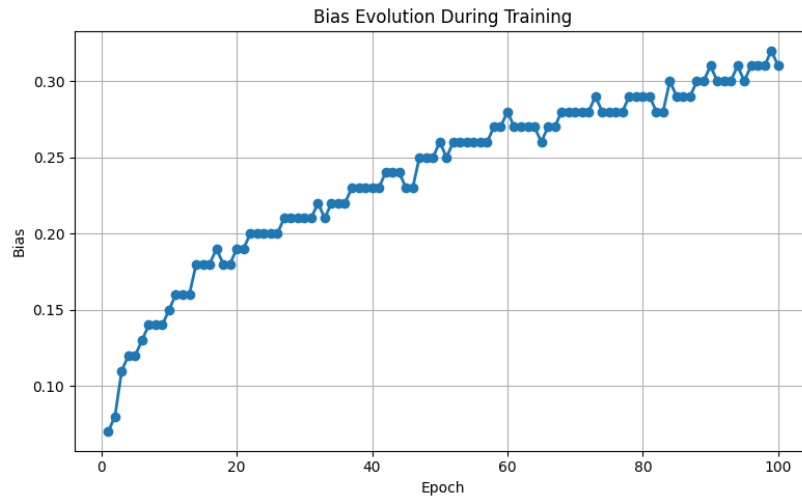


Figure 8: Bias Evaluation During Training

The bias value changes gradually during training before approaching a stable value. This demonstrates that the bias contributes significantly to shifting the decision boundary and improving classification accuracy.

9.9 Learning Rate Comparison

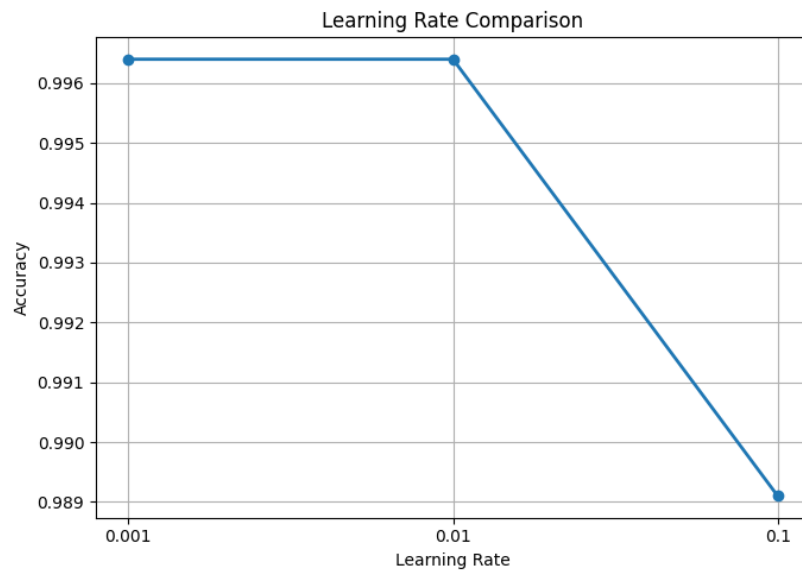


Figure 9: Learning Rate Comparison

The comparison of different learning rates indicates that both 0.001 and 0.01 achieved the highest testing accuracy of 99.64%. Increasing the learning rate to 0.1 resulted in a slight reduction in performance due to larger parameter updates during training.

10. Discussion

The implementation of the Single Layer Perceptron demonstrated that a simple linear classifier can effectively solve linearly separable binary classification problems. The Banknote Authentication dataset proved to be highly suitable for perceptron learning because the two classes become nearly separable after feature normalization.

The training error decreased considerably over successive epochs, indicating that the perceptron successfully learned an appropriate decision boundary. The gradual stabilization of both the weights and bias further confirmed convergence of the learning algorithm.

The evaluation metrics indicate excellent classification performance. The model achieved an Accuracy of 98.18%, Precision of 100.00%, Recall of 96.03% and an F1-Score of 97.98%. These values demonstrate that the implemented classifier generalized well on unseen testing samples.

The learning rate experiment showed that smaller learning rates (0.001 and 0.01) produced slightly better performance than a larger learning rate (0.1). Larger learning rates resulted in comparatively larger weight updates, producing a slight reduction in testing accuracy.

Although the Single Layer Perceptron performs well on linearly separable datasets, it cannot solve non-linearly separable problems such as XOR. Such problems require multilayer neural networks together with differentiable activation functions and backpropagation learning.

11. Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch using Python in Google Colab. The Banknote Authentication dataset was explored, preprocessed and normalized before training the classifier.

The perceptron learning algorithm successfully updated the weights and bias over multiple epochs, resulting in a well-trained binary classifier. The generated plots clearly illustrated the learning behaviour of the model through training error reduction, weight convergence, bias evaluation and learning-rate comparison.

The trained model achieved high classification performance with an Accuracy of 98.18%, demonstrating that the Single Layer Perceptron is an effective solution for linearly separable binary classification tasks. The experiment also provided practical understanding of activation functions, perceptron learning, parameter updates and performance evaluation metrics.

12. References

1. Frank Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, Psychological Review, 1958.
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
3. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Simon Haykin, *Neural Networks and Learning Machines*, Pearson Education, 2009.
5. UCI Machine Learning Repository, Banknote Authentication Dataset.
<https://archive.ics.uci.edu/dataset/267/banknote+authentication>
6. Scikit-learn Documentation.
<https://scikit-learn.org>